FemboyEngine2D

Application Guide

Практика ECS, оптимизация производства и SDK дистрибуции

0.4.0-alpha.1 · 2026-11-12

1. Для чего это руководство

FemboyEngine2D — двумерный движок, построенный вокруг data-oriented design. Это практическое дополнение к архитектурной спецификации: мы начинаем с играбельного 2D-шутера, показываем, как сущности собираются из независимых данных, а затем проводим тот же проект через кукинг текстур, GPU-батчинг, интеграцию магазинов, гибридную SDF-физику, погодный конвейер и шифрованный дистрибутивный архив.

Проект небольшой. Игрок, три архетипа врагов, босс с фазами, пули, подбираемые предметы и карточная доска — этого достаточно, чтобы показать все архитектурные правила. Каждый фрагмент кода компилируется против заголовков из Source/Engine; пути и имена типов настоящие.

- Аудитория: геймплей-программисты, технические дизайнеры и tools-инженеры.
- Требуемые знания: основы C++20, ECS на уровне chunked-реестра и один графический API.
- Не рассматривается: детали синхронизации Vulkan, сертификация консолей и юридическое подключение магазинов.

Note. Движок проверяется полным набором CTest. Наборы покрывают SDK-профиль и криптографию, мост магазинов, SDF-физику, стек рендера/погоды и путь data-driven префабов/DPI/темы. Перед тем как доверять локальному изменению, запустите ctest --test-dir build --output-on-failure.

2. Установка и воспроизводимый тулчейн

Скрипты установки разделены по ОС и группам ПО. C++-скрипты ставят зафиксированный toolchain и раскладывают точные исходники в third_party/. CMake предпочитает эти локальные копии, поэтому первый configure после установки не требует сети. Версии фиксируются скриптами и записываются в third_party/dependencies.lock.

```
# Windows 10/11 (run from a Visual Studio developer prompt)
SetupScripts\Windows\Setup.bat
SetupScripts\Windows\SetupCpp.bat
SetupScripts\Windows\SetupArchiver.bat
# optional vendor SDKs
SetupScripts\Windows\Setup.bat -WithVendorSdks -AcceptVendorLicenses

# Ubuntu / Debian
./SetupScripts/Linux/SetupCpp.sh
./SetupScripts/Linux/SetupArchiver.sh
# optional vendor SDKs
./SetupScripts/Linux/Setup.sh --with-vendor-sdks --accept-vendor-licenses

# macOS (double-click in Finder also works)
./SetupScripts/macOS/SetupCpp.command
./SetupScripts/macOS/SetupArchiver.command
# optional vendor SDKs
./SetupScripts/macOS/Setup.command --with-vendor-sdks --accept-vendor-licenses
```

Общие точки входа: SetupScripts/Windows/Setup.bat, SetupScripts/Linux/Setup.sh и SetupScripts/macOS/Setup.command.

Зафиксированные версии: SDL3 release-3.4.18, Vulkan-Headers v1.4.366, glslang 15.3.0, miniaudio 0.11.23, Box2D v3.2.0 и LZMA SDK 25.01. Setup по возможности ставит системные пакеты; используйте --skip-system (или -SkipSystem в Windows) на CI-образе, где уже есть CMake, Ninja, Clang/GCC и Vulkan loader.

```
cmake -S . -B build -G Ninja \
  -DCMAKE_BUILD_TYPE=RelWithDebInfo \
  -DCMAKE_C_COMPILER=clang-cl \
  -DCMAKE_CXX_COMPILER=clang-cl \
  -DFBY_BUILD_TESTS=ON
cmake --build build
ctest --test-dir build --output-on-failure
```

Компилятор — часть контракта: требуется C++26, эталонный тулчейн — Clang 23.1.4+, поддерживается GCC 16.2.0+. Сборка определяет FBY_HAS_MDSPAN, FBY_HAS_FORMAT и FBY_HAS_JTHREAD пробой компилятора, а не по номеру версии. Наличие LZMA SDK тоже проверяется; без него писатель .femarc использует детерминированный кодек FbyLZ и сообщает об этом в логе кукера.

> **Note.** Setup никогда не изменяет исходники. Можно удалить third_party/<name>-src и повторить запуск с --force; lock-файл перегенерируется, а CMake-кэш не содержит машинно-зависимого пути.

2.1 Редактор и собранная игра на Windows, Linux и macOS

Одни и те же пресеты собирают редактор, sandbox, инструменты ассетов и упаковщик игры. Windows использует clang-cl и Vulkan; Linux — clang++ и Vulkan; macOS — clang++ и Metal. Каждый пресет экспортирует compile_commands.json и оставляет тесты включёнными.

```
# Windows 10/11, from an LLVM + Visual Studio developer shell
cmake --preset windows-release
cmake --build --preset windows-release
ctest --preset windows-release

# Linux, from a shell with Clang, Ninja and Vulkan headers
cmake --preset linux-release
cmake --build --preset linux-release
ctest --preset linux-release

# macOS, from a shell with Xcode command-line tools and Homebrew
cmake --preset macos-release
cmake --build --preset macos-release
ctest --preset macos-release
```

Редактор находится в build-<platform>/bin/fby_editor (в Windows добавляется .exe). Build & Pack Game использует один и тот же редактор на всех трёх системах: готовит ассеты через fby_fempak, пишет AOT-модуль при наличии LogicGraph и раскладывает архивы в stores/ проекта. POSIX-путь релиза использует posix_spawn и возвращает код выхода дочернего процесса; Windows-путь — CreateProcess.

> **Note.** Windows — локально проверенный gate. Для Linux и macOS есть пресеты, платформенный код и CI-задания; считайте их первые запуски на runner’ах приёмочным gate, пока матрица не станет зелёной. RHI — Metal на macOS и Vulkan на Windows/Linux.

3. Как мыслить в ECS

ECS — это не другой синтаксис для иерархии классов. Это смена главного вопроса. В ООП вопрос звучит как «что это за объект?». В ECS вопросы другие: «какими данными владеет сущность?» и «какая система преобразует эти данные на этом тике?». Ответ на первый — набор типов компонентов; ответ на второй — система, обходящая все сущности с этим набором.

3.1 Правила, которые сохраняют скорость модели

- Компонент trivially copyable и standard layout: без string, vector и виртуальных методов. Transform2D, HealthComponent и WeaponComponent — значения, а не объекты.
- Поведение живёт в системе. У компонента нет метода Update(); система не хранит геймплейное состояние, принадлежащее сущности.
- Идентичность — это данные. Игрок не класс Player, а сущность с InputComponent, HealthComponent и WeaponComponent. Управлять можно любой сущностью с этими тремя компонентами.
- Вариативность — это данные. Архетип врага — строка таблицы определений, фаза босса — значение компонента, а не производный класс.
- Горячий путь не выделяет память. Ёмкость архетипов резервируется при загрузке; кадровая работа использует view, span и frame arena.

3.2 Словарь компонентов

Начните с небольшого словаря и не добавляйте компонент, пока двум системам не понадобится совместно использовать данные. Шутер использует следующие независимые компоненты.

```
// Game/Components.hpp - trivially copyable, no behaviour.
struct HealthComponent
{
    float current = 100.0f;
    float maximum = 100.0f;
    float invulnerableSeconds = 0.0f;
    std::uint32_t team = 0;
};

struct WeaponComponent
{
    std::uint64_t projectileArchetype = 0;
    float cooldownSeconds = 0.0f;
    float fireInterval = 0.12f;
    Vector2D muzzleOffset{18.0f, 0.0f};
    std::uint32_t pellets = 1;
};

struct InputComponent
{
    Vector2D move{};
    Vector2D aim{1.0f, 0.0f};
```

```
    std::uint32_t buttons = 0; // bit 0 fire, bit 1 dash, bit 2 interact
};

struct AIBehaviorComponent
{
    std::uint32_t behavior = 0; // 0 chase, 1 keep distance, 2 patrol, 3 boss
    Entity target = Fby::kNullEntity;
    float thinkSeconds = 0.0f;
    float preferredRange = 220.0f;
};

struct ProjectileComponent
{
    Entity owner = Fby::kNullEntity;
    Vector2D velocity{};
    float damage = 10.0f;
    float ttl = 4.0f;
    std::uint32_t pierce = 0;
};
```

Обратите внимание, чего здесь нет: базового класса Enemy, PlayerController, BossHealthBar. Это поведение и представление, собранные из тех же данных. Компилятор обеспечивает правило через static_assert(std::is_trivially_copyable_v<T>) в заголовке компонентов.

4. Сборка 2D-шутера через композицию

Игрок — это четыре компонента и трансформ. Его спавн — функция, а не конструктор. Эта же функция загружает префаб: сценический файл создаёт тот же набор компонентов через тот же код.

```
Entity SpawnPlayer(ecs::Registry& registry, Vector2D spawn)
{
    const Entity entity = registry.create();
    registry.emplace<Transform2D>(entity, Transform2D{spawn, {1.0f, 1.0f}, 0.0f});
    registry.emplace<Velocity>(entity);
    registry.emplace<HealthComponent>(entity, HealthComponent{100.0f, 100.0f,
0.0f, 0});
    registry.emplace<InputComponent>(entity, InputComponent{});
    registry.emplace<WeaponComponent>(entity, WeaponComponent{0x51A7u, 0.0f,
0.12f, {18.0f, 0.0f}, 1});
    registry.emplace<GameplayTagsComponent>(entity, GameplayTagsComponent{});
    return entity;
}

// Enemy archetypes are data. A boss is the same composition with different
values.
struct EnemyArchetype
{
    std::uint64_t id;
    float health;
    float speed;
    std::uint32_t behavior;
    float preferredRange;
    std::uint32_t weaponArchetype;
    Color32 tint;
};

Entity SpawnEnemy(ecs::Registry& registry, const EnemyArchetype& archetype,
Vector2D spawn)
{
    const Entity entity = registry.create();
    registry.emplace<Transform2D>(entity, Transform2D{spawn, {1.0f, 1.0f}, 0.0f});
    registry.emplace<Velocity>(entity);
    registry.emplace<HealthComponent>(entity, HealthComponent{archetype.health,
archetype.health, 0.0f, 1});
    AIBehaviorComponent behaviorComponent{};
    behaviorComponent.behavior = archetype.behavior;
    behaviorComponent.preferredRange = archetype.preferredRange;
    registry.emplace<AIBehaviorComponent>(entity, behaviorComponent);
    Sprite2D sprite{};
    sprite.Size = {32.0f, 32.0f};
    sprite.color = archetype.tint;
    registry.emplace<Sprite2D>(entity, sprite);
    return entity;
}
```

Мобы и боссы используют один путь спавна. «Босс» — это архетип с большим здоровьем, таблицей фаз и компонентом BossPhaseComponent, а не отдельный C++ тип. Новый враг — это строка данных и, если поведение новое, одна ветка в AISystem.

4.1 Системы по порядку

- InputSystem (PreUpdate): читает устройства в InputComponent. Ничего не двигает.
- AISystem (FixedUpdate): пишет желаемую скорость и прицел в тот же InputComponent, в который писал бы игрок. Это ключевой приём: одна система движения обслуживает игроков и врагов.
- MovementSystem (FixedUpdate): интегрирует Velocity в Transform2D; ограничивает границами уровня.
- WeaponSystem (FixedUpdate): перезарядка, спавн интентов снарядов через CommandBuffer.
- ProjectileSystem (FixedUpdate): интегрирует снаряды, TTL, запрос кандидатов broad-phase.
- DamageSystem (FixedUpdate, после ProjectileSystem): применяет попадания, неуязвимость и флаг смерти.
- CameraStackSystem (PostUpdate): следование, тряска, наклон и политика пост-эффектов.
- Extract/Render (Render): read-only проход, выдающий RenderItem и MeshRenderItem.

```
void AISystem(ecs::SystemContext& context)
{
    auto view = context.registry.view<AIBehaviorComponent, InputComponent,
Transform2D, HealthComponent>();
    for (const Entity entity : view)
    {
        const AIBehaviorComponent& behaviorComponent =
view.get<AIBehaviorComponent>(entity);
        const Transform2D& transform = view.get<Transform2D>(entity);
        const HealthComponent& health = view.get<HealthComponent>(entity);
        if (health.current <= 0.0f)
        {
            continue;
        }
        InputComponent& input = view.get<InputComponent>(entity);
        if (behaviorComponent.target == Fby::kNullEntity || !
context.registry.valid(behaviorComponent.target))
        {
            continue;
        }
        const Transform2D& target =
context.registry.get<Transform2D>(behaviorComponent.target);
        const Vector2D delta = target.Position - transform.Position;
        const float distance = std::sqrt(delta.LengthSquared());
        if (distance > behaviorComponent.preferredRange)
        {
            input.move = delta / std::max(distance, 0.001f);
        }
        else if (distance < behaviorComponent.preferredRange * 0.8f)
        {
            input.move = delta / std::max(distance, 0.001f) * -1.0f;
```

```
        }
        else
        {
            input.move = Vector2D{};
        }
        input.aim = delta / std::max(distance, 0.001f);
        input.buttons = distance < behaviorComponent.preferredRange * 1.2f ? 1u :
0u;
    }
}
```

Система небольшая. Здесь нет объекта-автомата и аллокации на врага; строка архетипа задаёт числа, и те же девять строк покрывают преследователя, турель и дальнего моба. Босс с фазами добавляет вторую маленькую систему, которая меняет preferredRange и поведение при переходе здоровья через порог.

4.2 Боссы без наследования

```
struct BossPhaseComponent
{
    std::uint32_t currentPhase = 0;
    std::uint32_t phaseCount = 0;
    float healthThresholds[4] = {0.66f, 0.33f, 0.12f, 0.0f};
    std::uint32_t behaviors[4] = {0u, 1u, 2u, 3u};
    float preferredRanges[4] = {220.0f, 160.0f, 90.0f, 40.0f};
};

void BossPhaseSystem(ecs::SystemContext& context)
{
    auto view = context.registry.view<BossPhaseComponent, HealthComponent,
AIBehaviorComponent>();
    for (const Entity entity : view)
    {
        BossPhaseComponent& phase = view.get<BossPhaseComponent>(entity);
        const HealthComponent& health = view.get<HealthComponent>(entity);
        AIBehaviorComponent& behaviorComponent =
view.get<AIBehaviorComponent>(entity);
        const float ratio = health.current / std::max(health.maximum, 0.001f);
        while (phase.currentPhase + 1 < phase.phaseCount &&
               ratio <= phase.healthThresholds[phase.currentPhase])
        {
            ++phase.currentPhase;
            behaviorComponent.behavior = phase.behaviors[phase.currentPhase];
            behaviorComponent.preferredRange =
phase.preferredRanges[phase.currentPhase];
        }
    }
}
```

Это вся иерархия боссов. «Огненный босс» и «босс-рывок» отличаются значениями компонентов и, в крайнем случае, таблицей спавна. Поведение, которое нельзя выразить

данными, заслуживает новой маленькой системы; оно почти никогда не оправдывает класс с vtable.

4.3 Спавн без фриза кадра

Структурные изменения во время обхода — классическая ошибка ECS. Шутер направляет любой спавн и удаление через CommandBuffer и применяет его на границе фазы. Бинарный CommandLog записывает те же команды для роллбэка, поэтому реплей видит идентичные id сущностей.

```
if (context.commands != nullptr)
{
    context.commands->Spawn(projectileEntity);
    context.commands->AddComponent(projectileEntity, ProjectileComponent{owner,
direction * 900.0f, 12.0f, 4.0f, 0});
}
// At the end of FixedUpdate the host calls commands->Apply(registry).
```

5. Планирование, детерминизм и тесты

SystemGraph — production-планировщик. Зависимости явные; Add() хранит фазу, указатель на функцию и битмаски after/before. Граф строится один раз, а не каждый кадр. Цикл или межфазная зависимость приводит к ошибке Build() и диагностируется, а не молча меняет порядок геймплея.

```
ecs::SystemGraph graph;
graph.Add(ecs::Phase::PreUpdate, "Input", &InputSystem);
graph.Add(ecs::Phase::FixedUpdate, "AI", &AISystem);
graph.Add(ecs::Phase::FixedUpdate, "Movement", &MovementSystem);
graph.Add(ecs::Phase::FixedUpdate, "Weapon", &WeaponSystem);
graph.Add(ecs::Phase::FixedUpdate, "Projectile", &ProjectileSystem);
graph.Add(ecs::Phase::FixedUpdate, "Damage", &DamageSystem);
graph.Add(ecs::Phase::PostUpdate, "Camera", &CameraTick);
if (!graph.Build())
{
    LogError("schedule", "system graph has a cycle or a cross-phase dependency");
}
```

Детерминизм — свойство данных, а не надежда. FixedUpdate получает постоянный dt, RNG — Pcg32 с сидом на запуск, структурные команды логируются. Snapshot v5 захватывает ECS плюс расширение мира (суставы, ткань, жидкость) и восстанавливает его с ремапом b2-тел. Тест реплея пересимулирует тот же лог ввода на чистом реестре и сравнивает контрольную сумму.

- Сначала headless: геймплейная система принимает SystemContext и не должна требовать окно или GPU.
- Тестируйте данные, а не пиксели: проверяйте значения компонентов и байты командного лога.
- Один исполняемый тест на задачу; без фреймворка и динамической регистрации.
- Запускайте весь набор через ctest; одно падение печатает файл и строку и возвращает ненулевой код.

5.1 Детерминированный ввод и репликация при создании игры

Ввод и репликацию проектируйте вместе, а не после готового геймплея. Маппер действий выдаёт одни и те же данные фиксированного тика на каждом пире; сетевой слой передаёт input frame, а не сырые SDL-события. Это контракт, который делает возможными предсказание и роллбэк.

```
InputSystem input;
const std::uint16_t Move = input.RegisterAction("Move", InputActionType::Axis2D);
const std::uint16_t Jump = input.RegisterAction("Jump", InputActionType::Bool);
const std::uint16_t Gameplay = input.RegisterContext("Gameplay", 0, false);

InputUserComponent user;
user.UserId = 0;
user.GamepadSlot = 0;
registry.emplace<InputUserComponent>(player, user);
input.BindUser(player, 0, 0);
input.PushContext(0, Gameplay);
```

```
// Once per fixed tick, before gameplay reads actions.
input.Update(window.Input(), tick, timestampNs, fixedDt);
const InputActionState& move = *input.Action(0, Move);
if (input.Action(0, Jump)->Triggered) { JumpPlayer(player); }
```

Локальный слой устройств может отличаться: клавиатура, мышь и геймпад идут в InputState разными путями. Симуляция видит только значения действий и события. Для сетевой игры упакуйте нужные значения в NetInputFrame (или собственную схему) и отправьте frame для владеющего игрока.

```
NetRollbackSession::Config net;
net.playerCount = 2;
net.localPlayer = 0;
net.rollbackTicks = 64;
NetRollbackSession session;
session.Initialize(registry, Ecs::SnapshotSchema::BuiltIns(), net);
session.SetCommandLog(&commands);
session.RegisterEntity(player, NetEntityId{1});

NetInputFrame frame;
frame.tick = tick;
frame.player = 0;
frame.moveX = static_cast<std::int16_t>(move.X * 32767.0f);
frame.buttons = input.Action(0, Jump)->Triggered ? 1u : 0u;
session.SubmitRemoteInput(0, frame);
```

- Авторитетная симуляция: сервер владеет тиком, сидом RNG и CommandLog. Клиенты предсказывают с тем же размером тика и теми же геймплейными системами.
- Регистрируйте геймплейные компоненты в SnapshotSchema до первого захвата; компонент вне схемы молча не реплицируется.
- Используйте стабильные NetEntityId для реплицируемых сущностей. Никогда не выводите их из runtime entity id, порядка спавна или адреса аллокатора.
- Реплицируйте input frames и структурные команды, а не сырые устройства или render-only значения.
- При коррекции восстановите снапшот до первого плохого тика, повторите исправленный ввод и сравните итоговую контрольную сумму.
- Проверяйте PIE на двух игроках, с искусственной задержкой, потерей пакетов и принудительным роллбэком до добавления контента.

Полезный порядок проекта: один игрок + InputSystem; два локальных пользователя; round-trip снапшота; серверно-авторитетное движение; затем взаимодействия, оружие и ИИ. Добавляйте поля репликации вместе с фичей-владельцем, а не одним поздним «networking pass». Если система читает wall-clock, delta кадра или render handle, она не готова к роллбэку.

6. Сжатие памяти и вызовов отрисовки

Вторая половина ECS-проекта — не геймплей, а то, что доходит до GPU. Основную работу делают два инструмента: запечённые блочно-сжатые текстуры и indirect-батчинг мешей. Оба настраиваются на проект и оба проверяемы через FrameStats.

6.1 Кукинг текстур: BC7 и ASTC

- BC7 сохраняет настоящий альфа-канал и остаётся десктопным вариантом по умолчанию. BC7 sRGB выбирается для цветных спрайтов; карта нормалей или маска использует линейный (UNORM) вариант.
- ASTC 4x4 — вариант качества для GPU с поддержкой ASTC; 6x6 — вариант размера. Выбирается в Project Settings -> Cooker -> Texture format.
- Кукер один раз при импорте строит полную цепочку мипов. Рантайм загружает блоки как есть; нет CPU-декодирования и повторного сжатия в рантайме.
- Единица батчинга — texture array. Держите спрайты одного материала в одном наборе слоёв; переключение слоя — это новый дескриптор, а не новый draw call.
- Никогда не поставляйте JPEG: кукер отвергает его по расширению и по magic. Lossy-источник делает поставленную текстуру отличной от утверждённой художником.

```
# project.femproj - the same values the panel writes
[render]
texture_format = "bc7-srgb"
max_texture_size = 2048

[build]
optimization = "O2"
lto = true
strip_debug_info = true
unity_build = true
native_arch = false
```

6.2 Multi-draw indirect батчинг

Спрайтовый путь уже использует один instanced-вызов на батч. Indirect-отрисовка важна на пути мешей: статическая геометрия один раз загружается в device-local арену и адресуется через MeshHandle; деформируемая геометрия стримится в transient ring; каждый draw — компактная запись MeshDrawData, а бэкенд проигрывает их одним drawIndexedIndirect на слой материала. CPU не биндит vertex buffer на каждый меш.

- Один материал на слой: новый слой — новый indirect-батч, но не переключение пайплайна.
- Статические объекты используют CreateMesh и MeshHandle; кадровая загрузка — потерянная пропускная способность PCIe.
- Скининговые меши скинятся одним compute-диспатчем в skinned vertex buffer, затем рисуются тем же indirect-путём.
- Следите за FrameStats: items, batches, drawCalls, meshIndirectCalls. Рост числа батчей с числом сущностей означает фрагментацию данных, а не медленный GPU.
- Render graph упорядочивает проходы; он не переупорядочивает draw внутри прохода. Держите список отрисовки отсортированным по pipeline/слою один раз за кадр.

Note. Процедурный путь Pink UI полностью убирает текстурную память: круги, скруглённые прямоугольники и карты вычисляются из 64-байтной записи экземпляра в pink_sdf.vert/frag. Доска из 200 слотов — это два instanced-вызова (карты и тени), а не 200 переключений текстуры.

7. Интеграция магазинов: Steam и Epic

PlatformService — это мост, а не обёртка над обёрткой. Движок владеет одной C-ABI таблицей функций, загруженной из провайдерской разделяемой библиотеки (fby_platform_steam или fby_platform_egs). Геймплей ставит команды в очередь; Pump() — единственное место вызова вендорского SDK, и он работает после записи кадра.

```
PlatformService platform;
std::string error;
if (!platform.Load(FbyPlatformKind_Steam, "fby_platform_steam.dll", &error))
{
    LogWarn("platform", "Steam provider unavailable: {}", error);
}
FbyPlatformInitDesc desc{};
desc.kind = FbyPlatformKind_Steam;
desc.applicationId = "480";
if (platform.Initialize(desc, &error))
{
    PlatformServiceCallbacks callbacks{};
    callbacks.onAchievement = &OnAchievement;
    platform.SetGameCallbacks(callbacks);
}
// Once per PreUpdate, after gameplay has enqueued this frame's intents:
platform.Pump();
```

7.1 Достижения как детерминированные команды

SteamAchievementSystem читает компоненты AchievementRequest, сортирует их по id достижения (чтобы порядок обхода ECS не менял лог), ставит одну команду на запрос и добавляет запись в PlatformCommandLog. Лог бинарный, с контрольной суммой и воспроизводимый; он отвечает на вопрос «почему это разблокировалось?», не доверяя дашборду магазина. EgsStoreSystem делает то же для облачных сохранений, с фиксированной staging-ареной и ограниченной очередью записи.

```
SteamAchievementSystem achievements;
achievements.Initialize(platform, commandLog);
EgsStoreSystem store;
store.Initialize(platform, commandLog);

ecs::SystemGraph graph;
SteamAchievementSystem::Register(graph, achievements);
EgsStoreSystem::Register(graph, store);
graph.Build();

// Gameplay only writes data; the systems do the rest.
AchievementRequest request{};
request.achievementId = 0xA17E;
request.Flags = AchievementPending;
std::strcpy(request.apiName, "ACH_FIRST_BLOOD");
registry.emplace<AchievementRequest>(player, request);

// Cloud save: stage once, then mark the component dirty.
```

```
store.StageWrite(0x51u, "slot1.sav", std::span<const std::byte>(saveBytes));
CloudSaveComponent save{};
save.saveId = 0x51u;
save.Flags = CloudSaveDirty;
std::strcpy(save.fileName, "slot1.sav");
registry.emplace<CloudSaveComponent>(player, save);
```

Кадр никогда не блокируется. Если провайдера нет, QueueAchievement возвращает false, запрос помечается failed, игра продолжает работать. C ABI версионирован; провайдер, собранный под другой ABI, отвергается до вызова. Steam резолвится через flat API в рантайме, поэтому игра не линкует steam_api; EGS использует EOS_Platform_Create с блобом опций от игры и может собираться с официальными заголовками при заданном FBY_EOS_SDK_INCLUDE_DIR.

8. Гибридная SDF-физика, лучи и пост-эффекты

Box2D решает контакты тело-тело. Ландшафт — не тело: это поле расстояний, которое рендерер уже сэмплирует. SdfCollisionSystem закрывает разрыв раз за фиксированный шаг. Он читает невладеющий SdfField над std::mdspan, билинейно сэмплирует его в повёрнутых углах коллайдера и центре, берёт градиент как нормаль, применяет импульс выталкивания с учётом массы и позиционную коррекцию, затем записывает исправленное состояние обратно в RigidBody2D.

```
// A height field is filled by the level loader once.
std::vector<float> samples(width * height);
for (std::uint32_t y = 0; y < height; ++y)
{
    for (std::uint32_t x = 0; x < width; ++x)
    {
        samples[y * width + x] = DistanceToTerrain(x, y); // negative inside
    }
}
SdfField field{};
field.samples = samples.data();
field.width = width;
field.height = height;
field.origin = levelOrigin;
field.cellSize = {2.0f, 2.0f};

SdfCollisionSystem sdf;
sdf.SetField(field);
sdf.Register(graph); // FixedUpdate, after the Box2D step
// Each Step produces up to 1024 SdfContact records for audio/VFX/decal reuse.
for (const SdfContact& contact : sdf.Contacts())
{
    SpawnImpactVfx(contact.point, contact.normal, contact.impulse);
}
```

8.1 SDF-тени и пост-эффекты, принадлежащие камере

SdfShadowPass марширует лучами по тому же полю в compute-шейдере и создаёт мягкую маску света; композит умножает на неё сцену. PostFxPass добавляет bloom-prefilter, разделяемое размытие и один композит для CRT-кривизны, scanline, виньетки, хроматической аберрации и зерна. Настройки живут в PostFxSettings на верхней записи CameraStack, поэтому крупный план терминала и геймплейная камера могут выглядеть по-разному без перестройки графа.

```
CameraStackSystem cameras;
CameraStackDesc camera{};
cameras.Push(registry, camera);

PostFxSettings terminal{};
terminal.Flags = PostFxCrt | PostFxVignette | PostFxBloom |
PostFxChromaticAberration;
terminal.crtCurvature = 0.12f;
terminal.scanlineStrength = 0.18f;
cameras.SetPostFx(terminal);
```

```
SdfShadowPass shadows;
SdfShadowDesc shadowDesc{};
shadowDesc.fieldWidth = width;
shadowDesc.fieldHeight = height;
shadowDesc.fieldCellSize = {2.0f, 2.0f};
shadows.Initialize(shadowDesc);
shadows.Build(graph);

PostFxPass postFx;
postFx.Build(graph, cameras);
```

Для UI SDF-путь полностью убирает текстурную память. pink_sdf.frag аналитически вычисляет круги, скруглённые прямоугольники, капсулы и кольца; сглаживание берётся из fwidth. SlotComponent и CardBoardComponent раскладывают карточную доску без сцены, а BuildCardBoardPrimitives упаковывает каждый слот в два экземпляра (карта и тень).

8.2 Движущиеся платформы, транспорт и колёса

Носитель — это система отсчёта трансформа, а не второй мир твёрдых тел. TransportGroupSystem подходит для лифтов, поездов и платформ: он считает дельты носителя до шага физики и обновляет локальные позы после него. Колесо — обычное динамическое тело с коллайдером; WheelSystem лениво создаёт wheel-джойнт Box2D и применяет настройки пружины, привода и тормоза колеса до шага.

```
TransportGroupSystem transport;
transport.CreateGroup(registry, elevator, 1);
transport.Attach(registry, player, 1, kCarrierModeSupport);

TransportSurface surface;
surface.Friction = 0.8f;
registry.emplace<TransportSurface>(elevator, surface);

Vehicle2D vehicle;
vehicle.MaxMotorTorque = 600.0f;
registry.emplace<Vehicle2D>(chassis, vehicle);

Wheel2D wheel;
wheel.Chassis = chassis;
wheel.LocalAnchor = {-20.0f, 4.0f};
wheel.Radius = 8.0f;
wheel.DriveWeight = 1.0f;
registry.emplace<Wheel2D>(wheelEntity, wheel);

WheelSystem wheels;
wheels.Initialize(physics);
wheels.Update(registry, physics, fixedDt);
physics.Step(registry, fixedDt);
```

9. Погодный конвейер

Погода — один компонент: WeatherContextComponent хранит плотность атмосферы, вязкость, множитель гравитации, облачность, плотность тумана, интенсивность дождя/снега/огня и глобальный ветер. WeatherSystem обновляет его в PreUpdate, выводит вектор ветра из fBm-шума, записывает результат обратно и передаёт в ClothSystem. Все последующие потребители используют один и тот же вектор, поэтому ткань, растительность и дождь не расходятся.

```
WeatherSystem weather;
weather.Initialize(WeatherDesc{0x51ED270Bu, Vector2D{1.0f, 0.0f}, 14.0f, 1.0f});
WeatherContextComponent context{};
context.atmosphereDensity = 0.82f;   // thinner air on an alien map
context.gravityScale = 0.65f;
context.windSpeed = 22.0f;
context.turbulence = 0.55f;
context.rainIntensity = 0.9f;
context.cloudCoverage = 0.7f;
context.fogDensity = 0.025f;
registry.emplace<WeatherContextComponent>(weatherEntity, context);

weather.Update(systemContext); // PreUpdate
// WindField() is now the shared block consumed by cloth, plants and GPU weather.
```

9.1 GPU-дождь со всплесками по SDF

WeatherPreconciliatedSystem не владеет массивом частиц на CPU. Он готовит WeatherGpuConstants, считает размер диспатча из интенсивности и записывает четыре прохода графа. weather_rain.comp владеет SSBO частиц, интегрирует ветер и гравитацию, сэмплирует глобальную SDF-сетку ландшафта и добавляет запись всплеска через atomicAdd, когда капля пересекает поверхность. Второй compute старит всплески, два graphics-прохода рисуют линии и кольца. Stats.cpuParticleReads по построению остаётся нулём.

```
WeatherPreconciliatedDesc rain{};
rain.particleCapacity = 262144;
rain.splashCapacity = 65536;
rain.areaWidth = 4096.0f;
rain.areaHeight = 2304.0f;

WeatherPreconciliatedSystem gpuRain;
gpuRain.Initialize(rain);
gpuRain.SetSdfField(field);
gpuRain.BuildRenderGraph(graph);

WeatherAtmospherePass atmosphere;
atmosphere.Build(graph, weather); // clouds, cloud shadows, fog, fire, composite

// Per frame, once:
gpuRain.Update(systemContext); // constants + dispatch size, zero particle reads
```

- Облака: двухоктавный simplex, адвектируемый разрешённым ветром, с формой из coverage и краем Worley.

- Тени облаков: то же поле, сэмплируемое сдвинутым ядром Пуассона; тени движутся с ветром.
- Туман: 24-шаговый вертикальный raymarch через экспоненциальный высотный туман с value-noise.
- Огонь/дым: simplex-пламя плюс медленный дым; интенсивность берётся из компонента погоды.
- Все четыре — compute-проходы, пишущие transient-текстуры; тяжёлые исходные текстуры не грузятся.
- Metal-ядро weather_rain.metal повторяет контракт Vulkan compute для Apple-таргетов.

10. Дистрибуция: .femarc, LZMA и AEAD

.fempak — mmap-дружественный рантайм-архив: без сжатия, детерминированные байты, сортированные пути. .femarc — дистрибутивная обёртка: LZMA на запись (или встроенный fallback FbyLZ) плюс AES-256-GCM или ChaCha20-Poly1305. Тег AEAD проверяется до того, как будет прочитан хоть один смещённый указатель. Читатель работает на буферах вызывающего, поэтому лаунчер расшифровывает в mapped-файл без аллокаций в куче.

```
fby_fempak secure --in build/release/assets.fempak --out archives/game.femarc \
  --entry game/assets.fempak --codec lzma --cipher aes \
  --key-env FBY_ARCHIVE_KEY
```

Ключ передаётся через переменную окружения или 64-hex аргумент; nonce генерируется из SecureRng, если не задан. Build идемпотентен для фиксированного набора ключ/nonce/записи: таблица сортирована, строковая таблица стабильна, сжатый поток детерминирован. Изменение одного байта даёт Open() = AuthenticationFailed, а не полуразобранный архив.

> **Note.** LZMA SDK опционален. Если third_party/lzma-sdk/LzmaLib.h существует, CMake собирает энкодер и декодер 7-Zip; иначе лог сборки сообщает 'FbyLZ fallback is active', и архив остаётся валидным, но больше. Поставка должна фиксировать и проверять один кодек, а не переключать его каждый запуск.

11. Плейбук производительности и CI

- Профилируйте fixed-step и кадровое время отдельно. Физика, ткань и AI относятся к первому; пост-эффекты и погода — ко второму.
- Держите fixed-step работу без аллокаций. Резервируйте ёмкость архетипов при загрузке сцены; для временных span используйте frame arena.
- Следите за тремя счётчиками до изменения кода: FrameStats.drawCalls, meshIndirectCalls и transient capacity. Рост drawCalls при неизменном числе элементов означает фрагментацию батчей.
- Для роллбэка и CI используйте профиль O2. Ofast может изменить результаты сравнения float и сломать детерминизм реплея; оставьте его для визуального прохода.
- LTO и unity build — только для релиза. Они сокращают время поставки, но увеличивают время линковки и делают инкрементальный кукинг неприятным.
- Никогда не допускайте вендорский колбэк внутрь геймплея. Ставьте в очередь, возвращайтесь, вызывайте Pump после кадра.

11.1 Чек-лист релиза

Запустите весь набор CTest, затем кукинг и secure-шаг на чистой машине. Проверьте, что игра запускается без Steam/EOS (отсутствие провайдера — предупреждение, а не падение), что изменённый .femarc отвергается и что набор SDF-физики проходит с точным релизным профилем. Запишите профиль и lock зависимостей в метаданные сборки.

```
ctest --test-dir build --output-on-failure
fby_fempak cook --root assets --out build/release/assets.fempak
fby_fempak secure --in build/release/assets.fempak --out archives/game.femarc \
  --codec lzma --cipher chacha --key-env FBY_ARCHIVE_KEY
# ReleaseCooker then stages archives/game.femarc and the plugin into
# stores/steam, stores/epic and stores/gog with a store.txt manifest.
# Launch the game with FBY_PLATFORM_LIB pointing at fby_platform_steam.dll
# or omit it and verify the NotAvailable path.
```

12. Материалы: автор, кук и привязка к мешу

Материал — небольшой граф, заменяющий фрагментный шейдер меша: базовый цвет, прозрачность и эмиссия, плюс математические узлы, выборки из атласа и 2.5D-помощники. Граф создаётся в редакторе, кукается на этапе сборки и загружается игрой. В игре GLSL не компилируется нигде.

Три артефакта

Файл	Кем создаётся	Кем загружается
`<name>.fgraph`	Панель редактора (Save .fgraph) или `EncodeMaterialGraph`	`fby_matc` и панель редактора (Load .fgraph)
`<name>.femmat`	`fby_matc` на шаге сборки	`Render::DecodeCookedMaterial` в игре
`MaterialPipelineCache`	Игра, один на устройство	Извлечение кадра через `MeshRenderItem`

Шаг сборки запускает `fby_matc <graph>.fgraph -o <graph>.femmat` (добавьте `--sprite` для контракта quad.vert). Инструмент валидирует граф, возвращает ненулевой код при ошибке и ничего не пишет, поэтому неудачный кук не оставит устаревший payload.

Привязка в игре

```
// Один раз, при старте устройства.
Render::MaterialPipelineCache Materials;

// Декодируйте payload и возьмите ссылку на пайплайн.
Render::CookedMaterialView Tinted;
if (!Render::DecodeCookedMaterial(InBytes, &Tinted))
{
    return false;
}
const Rhi::ShaderPipelineHandle TintedPipeline = Materials.Acquire(gpu, Tinted);

// Id означает использовать этот материал; значение по умолчанию — шейдер движка.
MeshRenderItem Item;
Item.materialPipeline = TintedPipeline.IsValid() ? TintedPipeline.id :
kDefaultMaterialPipeline;

// Остановка: отпускайте, пока устройство ещё живо.
Materials.Release(TintedPipeline);
Materials.Shutdown(gpu);
```

- Не кукайте на кадр или на загрузку уровня: `.femmat` — артефакт сборки, как сжатая текстура.
- Один кэш на устройство; `Acquire` безопасен на элемент и лишь увеличивает счётчик для того же хеша.

- Проверьте `SupportsRuntimeShaderPipelines()` при старте, а не угадывайте по невалидному хендлу.
- Спрайты кукаются отдельно: `fby_matc <graph> --sprite` собирает контракт quad.vert, `DrawOrder::MaterialBucket` сортирует спрайт в материальный прогон, а `SetMaterialPipelines` сопоставляет бакет с id кэшированного пайплайна.

Note. Справка: раздел 8 API reference перечисляет все типы и поля payload; глава 38 Complete Reference проходит кук, кэш и кадр; ADR 0002 фиксирует, почему рантайм не линкует glslang.

13. Приложение: карта компонентов и API

Используйте таблицу как стартовый чек-лист. Компонент, не помещающийся в одну строку, вероятно, это два компонента, а поведение, требующее новой строки, — это новая система, а не новый класс.

Компонент	Владелец	Пишет	Читает
Transform2D	любая видимая сущность	Movement, Physics	Render, Camera
Velocity	движущаяся сущность	AI, Movement	Physics, Movement
HealthComponent	получающая урон	DamageSystem	Damage, UI, BossPhase
WeaponComponent	вооружённая сущность	Loadout	WeaponSystem
InputComponent	игрок или AI	InputSystem / AISystem	Movement, Weapon
AIBehaviorComponent	AI-сущность	Spawner, BossPhase	AISystem
ProjectileComponent	пуля	WeaponSystem	Projectile, Damage
AchievementRequest	игрок	Геймплейный триггер	SteamAchievementSystem
CloudSaveComponent	аккаунт	Система сохранений	EgsStoreSystem
WeatherContextComponent	уровень	Дизайнер / кривая погоды	WeatherSystem, рендер
SlotComponent	карта	Логика колоды	CardBoardSystem, PinkSdf

11.1 Где смотреть в репозитории

- Source/Engine/ECS: Components.hpp, Schedule.hpp, SystemGraph.hpp, CommandLog.hpp.
- Source/Engine/Physics: SdfCollision.hpp/.cpp, Cloth.hpp/.cpp.
- Source/Engine/Platform: PlatformService, PlatformSystems; Source/Plugins/PlatformService: C ABI и провайдеры.
- Source/Engine/RHI: RenderGraph, RenderPasses, PostProcess, CameraStack, Shaders/.
- Source/Engine/Weather: Weather, WeatherPreconciliated.
- Source/Engine/UI: CardBoard, PinkSdf, PinkUI.
- Source/Engine/Assets: FemArchiver, FemCrypto, FempakWriter.
- Source/Tests: исполняемая спецификация; начните с SdkCryptoTest, PlatformBridgeTest, SdfPhysicsTest и RenderWeatherTest.
- Documentation/Tools/GenerateApplicationGuide.py: исходник этого документа.

Руководство генерируется из того же Python-файла, который пишет оба языка. Добавьте секцию в SECTIONS, запустите генератор — и Markdown с PDF останутся синхронными.